

《算法设计技巧与分析》

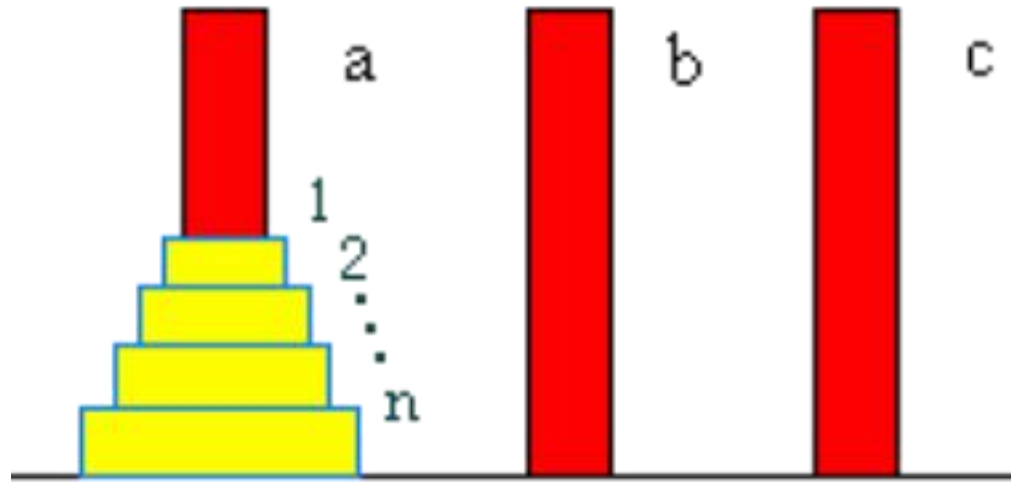
第4章 归纳法

朱友文

zhuyw@nuaa.edu.cn

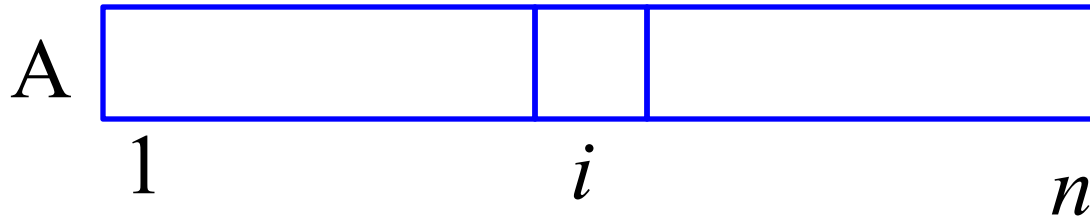
Hanoi塔问题

- 设A, B, C 是3 个塔座。开始时，在塔座A 上有一叠共 n 个圆盘，这些圆盘自下而上、由大到小地叠在一起。各圆盘从小到大编号为 $1, 2, \dots, n$
- 现要求将塔座A 上的这一叠圆盘移到塔座B 上，并仍按同样顺序叠置。在移动圆盘时应遵守以下移动规则：
 - 规则1：每次只能移动1个圆盘；
 - 规则2：任何时刻都不允许将较大的圆盘压在较小的圆盘之上；
 - 规则3：在满足移动规则1 和2 的前提下，可将圆盘移至A, B, C 中任一塔座上



```
void hanoi(int  $n$ , A, B, C) {  
    if ( $n > 0$ ) {  
        hanoi( $n-1$ , A, C, B); //A→C, B作辅助  
        move(A, B); // A→B  
        hanoi( $n-1$ , C, B, A); //C→B, A作辅助  
    }  
}
```

归纳法：插入排序



- 归纳法思路下的插入排序： $A[1, \dots, n]$
- 如果能对 $A[1, \dots, i]$ 进行排序；那么就可以对 $A[1, \dots, i+1]$ 进行排序
- $\text{sort}(A, 1, i)$
// $i > 1$, 实现对 $A[1, \dots, i]$ 的排序

归纳法：插入排序

➤ $\text{sort}(A, 1, i)$ // $i > 1$, 实现对 $A[1, \dots, i]$ 的排序

步骤: if $i > 1$ then

$\text{sort}(i-1)$

把 $A[i]$ 插入到有序序列 

endif

$x \leftarrow A[i]$

$j \leftarrow i - 1$

while $j > 0$ **and** $A[j] > x$

$A[j + 1] \leftarrow A[j]$

$j \leftarrow j - 1$

end while

$A[j + 1] \leftarrow x$

归纳法：插入排序

sort(i) // $i > 1$, 实现对 $A[1, \dots, i]$ 的排序

1. **if** $i > 1$ **then**
2. $x \leftarrow A[i]$
3. $\text{sort}(i - 1)$
4. $j \leftarrow i - 1$
5. **while** $j > 0$ **and** $A[j] > x$
6. $A[j + 1] \leftarrow A[j]$
7. $j \leftarrow j - 1$
8. **end while**
9. $A[j + 1] \leftarrow x$
10. **end if**

归纳法： 插入排序

➤ 时间复杂度： 元素赋值次数记为 $C(n)$, 那么

➤ $C(n) = C(n-1) + (n-1)$

4.2 寻找多数元素

➤ 多数元素：出现次数大余 $n/2$ 的元素

➤ 直观的方法

- 蛮力法
- 先排序，再判断

➤ 结论5.1

观察结论 5.1 在原序列中去除两个不同的元素后，那么在原序列中的多数元素在新序列中还是多数元素。

4.2 寻找多数元素

3, 5, 6, 3, 3, 3, ~~7~~

观察结论 5.1 在原序列中去除两个不同的元素后，那么在原序列中的多数元素在新序列中还是多数元素。

4.2 寻找多数元素

➤ 实现过程

- 找出可能是多数元素的值
- 判断这个可能值是否是多数元素(给定一个值 c , 判定是否是多数元素)

count $\leftarrow$ 0

for $j \leftarrow 1$ **to** n

if $A[j] = c$ **then** *count* $\leftarrow$ *count* + 1

end for

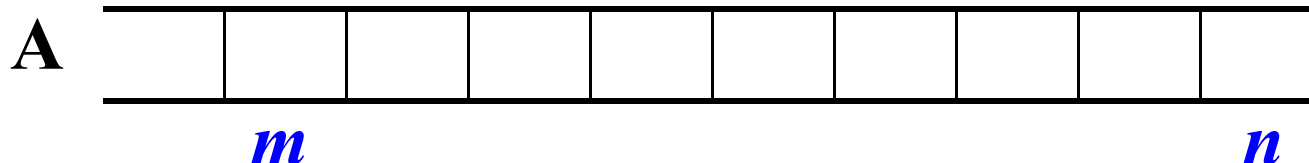
if *count* > $\lfloor n/2 \rfloor$ **then return** c

else return *none*

4.2 寻找多数元素

怎么找候选者？

➤ 找出 $A[m, \dots, n]$ 可能是多数元素的值（候选者）



(1) 如果 $A[m] \neq A[m+1]$,

则，可以排除它们俩，从 $A[m+2, \dots, n]$ 找多数元素

(2) 如果 $A[m] = A[m+1]$,

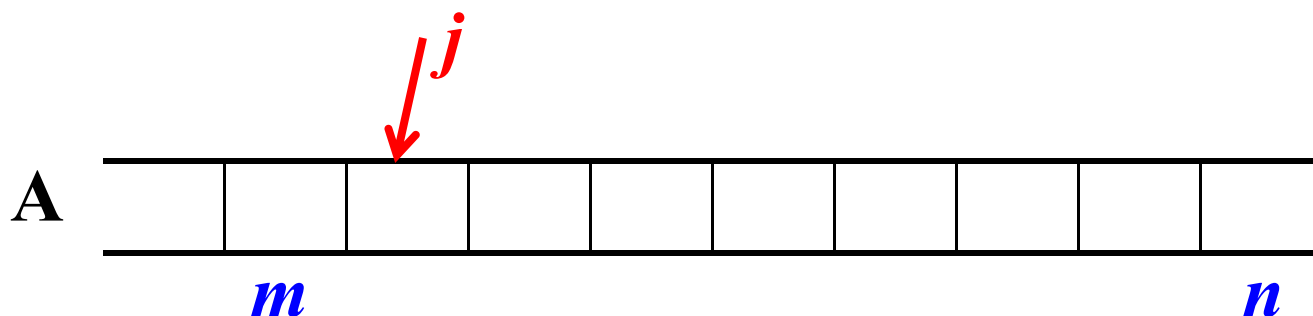
则，继续考察 $A[m+2]$ 、 $A[m+3]$,

(3) 依次类推.....

4.2 寻找多数元素

怎么找候选者?

➤ 找出 $A[m, \dots, n]$ 可能是多数元素的值 (候选者)



$count$ 初始值设为1;

如果 $A[j] = A[m]$, 则 $count++$; 否则 $count--$;

当 $count$ 减到0时, 说明了删掉若干两两不等的元素对,
则可以从数组 A 的后续元素中继续寻找候选者

4.2 寻找多数元素

➤ 找出 $A[m, \dots, n]$ 可能是多数元素的值（候选者）

```
candidate(m) //找出A[m,...,n]可能是多数元素的值
{
    j = m; c = A[m]; count=1;
    while( j<n 且 count>0) {
        j++;
        if(A[ j]=c) count++;
        else count--;
    } //endwhile
    if(j=n) return c;
    else candidate( j+1);
}
```

4.2 寻找多数元素

算法 5.9 MAJORITY

输入： n 个元素的数组 $A[1 \cdots n]$ 。

输出：若存在多数元素，则输出；

找出可能是多数元素的值

1. $c \leftarrow \text{candidate}(1)$

2. $\text{count} \leftarrow 0$

3. **for** $j \leftarrow 1$ **to** n

4. **if** $A[j] = c$ **then** $\text{count} \leftarrow \text{count} + 1$

5. **end for**

6. **if** $\text{count} > \lfloor n/2 \rfloor$ **then return** c

7. **else return** none

判断这个可能值是不是多数元素

➤ 时间复杂度 $O(n)$

4.2 寻找多数元素

如果不用递归，怎么实现？

4.2 寻找多数元素

如果不用递归，怎么实现？

过程 *candidate*(*m*)

1. $j \leftarrow m; c \leftarrow A[m]; \text{count} \leftarrow 1$
2. **while** $j < n$ **and** $\text{count} > 0$
3. $j \leftarrow j + 1$
4. **if** $A[j] = c$ **then** $\text{count} \leftarrow \text{count} + 1$
5. **else** $\text{count} \leftarrow \text{count} - 1$
6. **end while**
7. **if** $j = n$ **then return** c {见练习 5.31 和练习 5.32}
8. **else return** $\text{candidate}(j + 1)$

4.2 寻找多数元素

```
candidate(m) //找出A[m,...,n]可能是多数元素的值
{
  while(m ≤ n){
    j = m; c = A[m]; count=1;
    while( j < n 且 count > 0) {
      j++;
      if(A[j]=c) count++; else count--;
    } //endwhile
    if(j=n) return c;
    else m = j+1;
  } //endwhile
}
```

4.3 整数幂

➤ 计算 x （实数）的 n 次幂（ n 是整数）

$$x^n = ?$$

4.3 整数幂

➤ 你怎么计算 x 的 n 次幂?

➤ 两种计算路径(以3的8次方为例):

(1) 每次乘3

$3 \rightarrow 3^2 \rightarrow 3^3 \rightarrow 3^4 \rightarrow 3^5 \rightarrow 3^6 \rightarrow 3^7 \rightarrow 3^8$

7次($n-1$ 次)
乘法

(2) 平方

$3 \rightarrow 3^2 \rightarrow 3^4 \rightarrow 3^8$

3次($\log n$ 次)
乘法

如果 n 不是2
的整数次幂
呢?

4.3 整数幂

➤ 计算 x (实数) 的 n 次幂 (n 是整数)

➤ 实现思路:

如果能计算出 x 的 b 次幂, $b = \lfloor \frac{n}{2} \rfloor$

那么就可以计算出 x 的 n 次幂

$$t = x^b,$$

如果 n 为偶数, $y = t^2$

否则, $y = x * t^2$

4.3 整数幂

➤ 具体步骤

➤ 终止条件

➤ 递归过程

过程 $power(x, m)$ {计算 x^m }

1. **if** $m = 0$ **then** $y \leftarrow 1$

2. **else**

3. $y \leftarrow power(x, \lfloor m/2 \rfloor)$

4. $y \leftarrow y^2$

5. **if** m 为奇数 **then** $y \leftarrow xy$

6. **end if**

7. **return** y

4.3 整数幂

➤ 时间复杂度 $O(\log n)$

计算代价满足: $C(n) \leq C(n/2) + 2$

过程 $power(x, m)$ {计算 x^m }

1. **if** $m = 0$ **then** $y \leftarrow 1$

2. **else**

3. $y \leftarrow power(x, \lfloor m/2 \rfloor)$

4. $y \leftarrow y^2$

5. **if** m 为奇数 **then** $y \leftarrow xy$

6. **end if**

7. **return** y

4.3 整数幂

► 循环迭代的实现方法：主要思路

任意整数 n ，都可拆为若干个2的整数次幂之和

比如 $9 = 2^3 + 2^0$, $13 = 2^3 + 2^2 + 2^0$

实际上，如果 n 化为2进制为 $d_k d_{k-1} \cdots d_0$ ，那么

$$n = d_k * 2^k + d_{k-1} * 2^{k-1} + \cdots + d_i * 2^i + \cdots + d_0 * 2^0,$$

进一步，

$$x^n = x^{d_k 2^k} * \cdots * x^{d_i 2^i} * \cdots * x^{d_0 2^0}$$

4.3 整数幂

➤ 循环迭代的实现方法：方法一

$$x^n = x^{d_k 2^k} * \dots * x^{d_i 2^i} * \dots * x^{d_0 2^0}$$

如果 $d_i=0$, 那么 $x^{d_i 2^i}=1$;

如果 $d_i=1$, 那么 $x^{d_i 2^i} = ((x^2)^{2^{\dots}})^2$

i 次

$$x \rightarrow x^2 \rightarrow x^4 \rightarrow x^8 \rightarrow \dots \rightarrow x^{2^{k-1}} \rightarrow x^{2^k}$$

4.3 整数幂

➤ 循环迭代的实现方法：方法一

$$x^n = x^{d_k 2^k} * \dots * x^{d_i 2^i} * \dots * x^{d_0 2^0}$$

如果 $d_i=0$, 那么 $x^{d_i 2^i}=1$;

如果 $d_i=1$, 那么 $x^{d_i 2^i} = ((x^2)^{2 \dots})^2$

*i*次

```
y = 1;
for(i = k; i >= 0; i--) {
    y = y * y; //平方
    if(d_i = 1) y = y * x;
}
```

➤ 时间复杂度 $O(\log n)$

4.3 整数幂

➤ 循环迭代的实现方法：方法二

$$x^n = x^{d_k 2^k} * \dots * x^{d_i 2^i} * \dots * x^{d_0 2^0}$$

如果 $d_i=0$, 那么 $x^{d_i 2^i}=1$;

如果 $d_i=1$, 那么 $x^{d_i 2^i} = (x^{2^{i-1}})^2$

➤ 时间复杂度 $O(\log n)$

➤ 空间复杂度 $O(1)$

```
y = 1; t = n; p = x;
while( t > 0) {
    if(t mod 2) y = y * p;
    t = ⌊t/2⌋;
    p = p * p; //平方
}
return y;
```

4.3 整数幂： 考试题(16年)

本题分数	10
得 分	

二、斐波那契数列满足如下条件：

$$\begin{cases} F_0 = 0, F_1 = 1, \\ \text{当 } n \geq 2 \text{ 时, } F_n = F_{n-1} + F_{n-2} \end{cases}$$
，假设矩阵 $A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$ ，那么

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = A \begin{bmatrix} F_{n-1} \\ F_{n-2} \end{bmatrix} = A^2 \begin{bmatrix} F_{n-2} \\ F_{n-3} \end{bmatrix} = \dots = A^{n-1} \begin{bmatrix} F_1 \\ F_0 \end{bmatrix}, \text{ 即 } F_n \text{ 等于 } A^{n-1} \text{ 左上角的元素值。}$$

令 $m=n^3$ ，利用以上结论，能在 $\Theta(\log n)$ 时间求出斐波那契数列中的 F_m 吗？如果能，给出相应的算法；如果不能，说明理由。

```
y = 1; t = n; p = x;
while( t > 0 ) {
    if(t mod 2) y = y*p;
    t = t / 2;
    p = p * p; //平方
}
return y;
```

A^{m-1}

```
Y = I2*2; t = m-1; P = A;
while( t > 0 ) {
    if(t mod 2) Y = Y*P;
    t = ⌊t/2⌋;
    P = P * P;
}
return Y11;
```

4.4 多项式求值 (Horner)

➤ 问题描述&方法一

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

对于任意 i , 那么 $a_i x^i = a_i * x * x * \cdots * x$ (i 次)

```
y = an;
```

```
for(i = n-1; i >= 0 ; i--)
```

```
    y = y * x + ai ;
```

算法 5.6 HORNER

输入: $n+2$ 个实数 $a_0, a_1, \dots, a_n$ 和 x 的序列。

输出: $P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$

1. $p \leftarrow a_n$
2. for $j \leftarrow 1$ to n
3. $p \leftarrow xp + a_{n-j}$
4. end for
5. return p

➤ 时间复杂度 $O(n)$, 空间复杂度 $O(1)$

4.4 多项式求值 (Horner)

➤ 方法二

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

```
y = 1; t = n; p = x;  
while( t > 0) {  
    if(t mod 2) y = y*p;  
    t = t / 2 ; //平方  
    p = p * p;  
}  
return y;
```

```
y = a0;  
p = 1;  
for(i = 1; i ≤ n; i++){  
    p = p * x;  
    y = y + ai * p ;  
}  
return y;
```

➤ 时间复杂度 $O(n)$, 空间复杂度 $O(1)$

练习 4.3.2

令 $A[1\dots n]$ 是含 n 个整数的有序数组， x 是给定整数，请设计一个算法，判断数组 A 中是否存在两个数等于 x

三种不同的算法

- 时间复杂度为 $O(n^2)$ 的算法
- 时间复杂度为 $O(n\log n)$ 的算法
- 时间复杂度为 $O(n)$ 的算法